**P.Vishwa Nandan(192473015)**

## TASK 1 – Understand the Problem and Identify the Objects

### 1. Problem Description

The Campus Smart-Parking System is designed to manage vehicle parking within a university campus. It supports different vehicle types, including two-wheelers, cars and electric vehicles.

When a vehicle enters the parking facility, the system searches for an available parking slot and allocates it. It generates a unique parking ticket and records the entry time.

When the vehicle leaves, the system calculates the parking duration and parking fee. Students and staff with valid monthly passes can park free of charge, whereas visitors must pay the applicable fee.

The system also supports multiple floors and electric vehicle charging slots.

### 2. Inputs

The system receives the following inputs:

1. Vehicle registration number.
2. Vehicle type.
3. User category (Student, Staff or Visitor).
4. Vehicle entry time.
5. Vehicle exit time.
6. Monthly pass validity.
7. Number of floors and parking slots.

### 3. Outputs

The system produces the following outputs:

1. Allocated floor number.
2. Allocated parking slot number.
3. Unique parking ticket number.
4. Vehicle entry time.
5. Total parking duration.
6. Calculated parking fee.
7. Parking slot availability status.

8. Charging fee for electric vehicles.

**4. Candidate Classes and Their Responsibilities**

The following candidate classes are identified from the parking problem.

**1. ParkingFacility**

Manages the complete parking facility. It checks slot availability, allocates slots and releases occupied slots.

**2. ParkingFloor**

Represents an individual parking floor and maintains its parking slots.

**3. ParkingSlot**

Stores the slot number and occupancy status. It ensures that only one vehicle occupies a slot at a time.

**4. Vehicle**

Represents a general vehicle and stores common information such as registration number and vehicle type.

**5. TwoWheeler**

Represents two-wheeler vehicles and defines their specific parking fee.

**6. Car**

Represents car vehicles and defines their parking fee.

**7. ElectricCar**

Represents electric vehicles and provides additional charging functionality.

**8. ParkingTicket**

Stores the unique ticket number, vehicle details and entry time.

**9. ParkingPass**

Maintains monthly pass information and checks whether a student or staff member has a valid pass.

**5. Candidate Methods**

The following methods are required:

- allocateSlot() – Allocates an available parking slot.
- releaseSlot() – Releases an occupied parking slot.
- generateTicket() – Generates a unique parking ticket.
- recordEntry() – Records the vehicle entry time.
- calculateFee() – Calculates the parking fee.
- findAvailableSlot() – Searches for an empty slot.
- validatePass() – Checks monthly pass validity.
- chargingFee() – Calculates the EV charging fee.

These methods represent the main actions performed by the parking system.

**6. Business Rules**

The parking system must follow these rules:

1. A parking slot can hold only one vehicle at a time.
2. A vehicle cannot be allocated to an already occupied slot.
3. Every parking ticket must have a unique ticket number.
4. A ticket number must never be reused.
5. A vehicle must have a valid registration number.
6. A vehicle must be assigned a suitable parking slot.
7. An occupied slot must be released when the vehicle leaves.
8. Students and staff with valid monthly passes can park free of charge.
9. Visitors must pay the applicable parking fee.
10. Parking duration and parking fees cannot be negative.

**7. Assumptions**

1. Every parking slot can accommodate only one vehicle.
2. Each vehicle has a unique registration number.
3. The parking facility can contain multiple floors.
4. Parking charges depend on vehicle type and parking duration.
5. Electric vehicles may use special charging slots.
6. The parking facility uses a unique ticket counter starting from 1.
7. Parking rates and fee slabs are predefined by the university.

## 8. Alternative Concept and Complexity Analysis

In the original design, every vehicle is stored as an object of the Vehicle class.

Alternatively, vehicles can be stored using parallel arrays:

- regNo[] – Stores vehicle registration numbers.
- type[] – Stores vehicle types.
- entryTime[] – Stores vehicle entry times.

**Finding a vehicle:**

In the object-oriented approach, the system searches through Vehicle objects using the registration number.

In the parallel-array approach, the system searches through the regNo[] array and uses the same index to access the corresponding vehicle type and entry time.

Both approaches require O(n) time for a linear search, where n represents the number of vehicles.

**Adding a new attribute:**

In the object-oriented approach, a new attribute can be added directly to the Vehicle class.

In the parallel-array approach, a new array must be created and maintained with the same index positions.

**Comparison:**

The object-oriented approach is easier to maintain and extend because vehicle information and behaviour are grouped inside objects.

Parallel arrays may require less object-related memory overhead, but maintaining multiple arrays with matching indexes can become difficult.

## 9. Conclusion

The object-oriented approach is suitable for the Campus Smart-Parking System because it provides better organization, reusability, maintainability and future extensibility.